Laboratorio evolutivo 07

Modernización de Digital Café Luna en AWS

Cloud Computing Posgrado · Semana 7 · Cuenta AWS estándar

Duración	Modalidad	Entorno	Carácter
40 min hands-on	Equipos de 4	Cuenta AWS estándar	Formativo; con evidencia

1. Propósito y resultado

Demostrar que un workload corto y orientado a eventos puede ejecutarse sin administrar una VM dedicada, pero que la responsabilidad operacional no desaparece: cambia de frontera.

Al finalizar, el equipo habrá relacionado un objeto nuevo en S3 con una invocación de Lambda y un log verificable, y podrá comparar qué responsabilidades se transfieren a la plataforma y cuáles siguen perteneciendo al equipo.

Hipótesis práctica

Para una reacción corta, intermitente y orientada a eventos, una función serverless puede eliminar responsabilidades de infraestructura que una VM exigiría mantener, sin eliminar responsabilidades de permisos, código, eventos, logs y costo.

Objetivos de aprendizaje

- Reconstruir productor → evento → consumidor → evidencia sin depender de una VM dedicada.
- Validar que el execution role de Lambda solo necesita permisos para registrar logs en este ejercicio.
- Demostrar comportamiento mediante objeto, invocación y log correlacionados.
- Distinguir responsabilidades transferidas a la plataforma de responsabilidades que permanecen en el equipo.
- Realizar cleanup completo sin modificar el baseline S01.

Ruta de la sesión

Minuto sesión	Bloque	Salida
0–10	Recap + preflight común	Baseline intacto · nombres y roles claros
10–25	S3 + log group + role + Lambda + trigger	Integración preparada
25–35	Evento + CloudWatch Logs	Objeto → invocación → log
35–50	Comparación + evidencia del lab	Trade-off explicado
50–75	Checkpoint Lab 2	Defensas de 4 equipos
75–85	Cleanup	Recursos S07 eliminados
85–90	Cierre hacia S08	Decisión arquitectónica articulada

Arquitectura objetivo

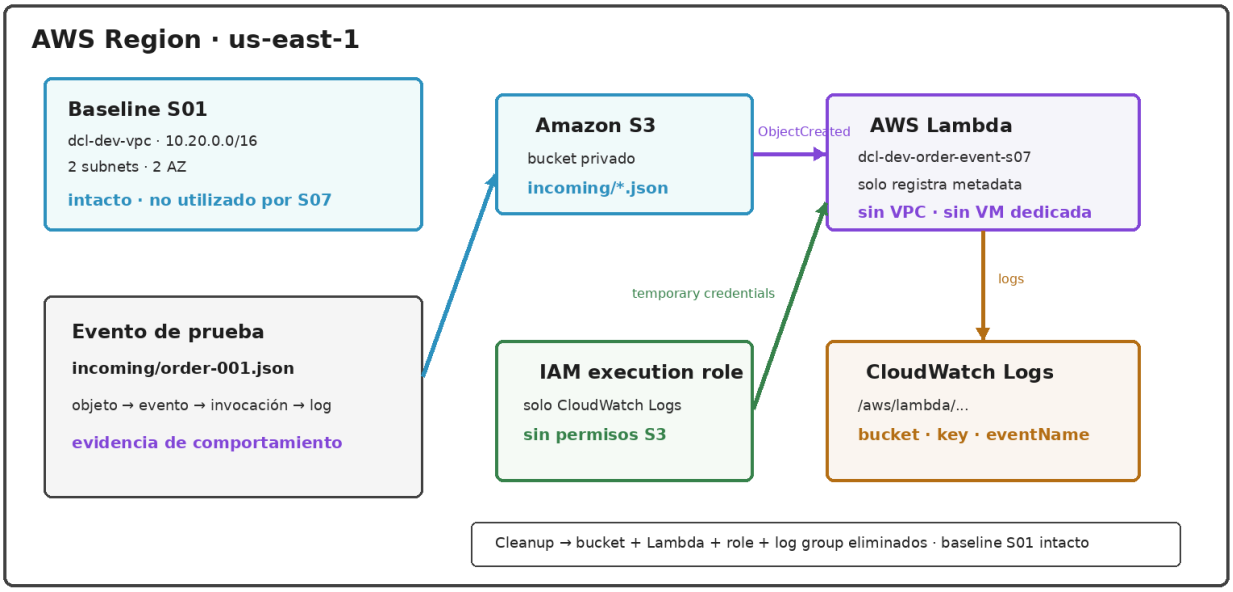


Figura 1. S07 usa una integración administrada fuera de la VPC: el baseline S01 permanece intacto y no forma parte del camino de ejecución.

2. Baseline y preflight

Qué hacemos: Confirmar Región, baseline persistente y permisos normales de la cuenta antes de crear recursos temporales.

Por qué: S07 moderniza la forma de ejecución; no necesita modificar red, subnets, routing ni conectividad solo porque los labs anteriores sí utilizaron la VPC.

Elemento	Valor / decisión
Región	us-east-1
Baseline	dcl-dev-vpc · 10.20.0.0/16 · dos subnets en AZ distintas
Uso de VPC en S07	Ninguno; se conserva intacta
Bucket	dcl-s07-teamXX-<account-id> · privado · us-east-1
Lambda	dcl-dev-order-event-s07
Execution role	dcl-dev-role-lambda-s07 · solo permisos de CloudWatch Logs
Log group	/aws/lambda/dcl-dev-order-event-s07 · retención 1 día

1. Confirmen que la consola está en us-east-1 y registren Account ID para construir un nombre de bucket único.
2. Confirmen que no existen recursos S07 de una ejecución anterior con los mismos nombres.
3. La identidad de la consola debe poder usar S3, Lambda, IAM y CloudWatch Logs. No creen access keys como workaround.
4. No modifiquen VPC, subnets, route tables, security groups ni conectividad del baseline.

Validación: los recursos persistentes S01 están intactos y el equipo tiene nombres únicos definidos para S07.

3. Prueba 1 — preparar la integración

Qué hacemos: Crear el productor (S3), el consumidor (Lambda), su identidad de ejecución y el destino de logs antes de generar el evento.

Por qué: La integración solo es demostrable si podemos reconstruir quién produce el evento, quién lo consume y dónde queda evidencia.

Paso A — bucket y log group

5. En S3 creen un bucket General purpose privado en us-east-1 con nombre dcl-s07-teamXX-<account-id>. Mantengan Block Public Access habilitado.
6. En CloudWatch > Log groups creen /aws/lambda/dcl-dev-order-event-s07 y configuren retención = 1 día.
7. No carguen todavía incoming/order-001.json; primero completaremos la integración.

Paso B — execution role mínimo

8. En IAM > Roles creen dcl-dev-role-lambda-s07 con trusted entity = AWS service y use case = Lambda.
9. No adjunten políticas de S3. Agreguen una inline policy con las dos acciones siguientes, acotada al log group precreado.

```
{
  "Version": "2012-10-17",
  "Statement": [{
    "Effect": "Allow",
    "Action": ["logs:CreateLogStream", "logs:PutLogEvents"],
    "Resource": "arn:aws:logs:us-east-1:<ACCOUNT_ID>:log-group:/aws/lambda/dcl-dev-order-event-s07:*"
  }]
}
```

OJO TÉCNICO: El log group se crea antes para evitar conceder logs:CreateLogGroup. La función no lee el objeto; por eso su execution role no necesita permisos S3.

Paso C — función Lambda

10. En Lambda creen dcl-dev-order-event-s07 con runtime Python 3.14, arquitectura x86_64 y Use an existing role = dcl-dev-role-lambda-s07.
11. Reemplacen el código inicial por el bloque siguiente y seleccionen Deploy. No se evalúa Python; el código solo registra metadata del evento.

```
import json
import logging
from urllib.parse import unquote_plus

logger = logging.getLogger()
logger.setLevel(logging.INFO)

def lambda_handler(event, context):
    for record in event.get('Records', []):
        payload = {
            'bucket': record['s3']['bucket']['name'],
            'key': unquote_plus(record['s3']['object']['key']),
            'eventName': record.get('eventName'),
            'eventTime': record.get('eventTime'),
            'requestId': context.aws_request_id
        }
        logger.info(json.dumps(payload))
    return {'records': len(event.get('Records', []))}
```

Prueba 1 — trigger y validación

Paso D — configurar ObjectCreated

12. En la función Lambda, seleccionen Add trigger > S3.
13. Bucket = el bucket S07 del equipo. Event type = All object create events (equivale a s3:ObjectCreated:*).
14. Prefix = incoming/ y Suffix = .json.
15. Confirmen que bucket y función están en us-east-1 y creen el trigger.
16. En Configuration > Permissions validen que la resource-based policy de Lambda contiene permiso para que S3 invoque esta función desde el bucket del equipo.

Validación: el bucket permanece privado; el trigger solo aplica a incoming/*.json; la función usa el role mínimo; el baseline S01 no cambió.

> **Regla crítica:** La función no escribe objetos dentro de incoming/ ni en ningún prefijo que vuelva a disparar el mismo trigger. En este lab la función solo registra metadata.

4. Prueba 2 — producir el evento

Qué hacemos: Crear exactamente un objeto dentro del prefijo que activa la notificación.

Por qué: El comportamiento debe empezar en una condición observable del productor, no en una invocación manual de Lambda.

Creen localmente un archivo order-001.json con el payload mínimo siguiente y súbanlo al bucket con la key incoming/order-001.json.

```
{
  "orderId": "order-001",
  "source": "lab-s07",
  "action": "created"
}
```

Validación: el objeto existe con key exacta incoming/order-001.json y fue creado después de que el trigger quedó configurado.

5. Prueba 3 — demostrar comportamiento

Qué hacemos: Localizar la ejecución asociada al objeto anterior en CloudWatch Logs.

Por qué: La evidencia útil no es “Lambda existe”; debe relacionar el mismo objeto con el evento, la invocación y el log.

17. Abrir CloudWatch > Logs > Log groups > /aws/lambda/dcl-dev-order-event-s07.
18. Abrir el log stream más reciente y localizar una línea JSON emitida por el código.
19. Confirmar bucket, key=incoming/order-001.json, eventName y requestId/eventTime.
20. Si el log tarda en aparecer, continúen con la comparación operacional de la página siguiente y vuelvan a revisar. No cambien permisos sin evidencia.

Validación: una misma evidencia permite reconstruir objeto → ObjectCreated → invocación → log.

> **Nota técnica:** Las notificaciones S3 se diseñan para entrega at-least-once. Una notificación duplicada ocasional no implica automáticamente que el lab falló; en producción habría que considerar idempotencia y ordering, pero no los profundizaremos aquí.

6. Prueba 4 — comparación operacional

Qué hacemos: Comparar esta reacción event-driven con una implementación equivalente que mantuviera una EC2 esperando eventos.

Por qué: Modernizar no significa escoger la tecnología más nueva; significa mover responsabilidades cuando las restricciones del workload lo justifican.

Responsabilidad	EC2 — reacción equivalente	Lambda — este lab
Servidor		
Sistema operativo		
Runtime		
Scaling / capacidad		
Disponibilidad de infraestructura		
IAM / permisos		
Código		
Logs / evidencia		
Costo cuando no hay ejecución		

Conclusión del equipo

¿Qué responsabilidades fueron transferidas a la plataforma y cuáles siguen perteneciendo al equipo?

Pregunta de decisión

> **Responder con una restricción, no con una etiqueta:** ¿Qué restricción de este workload hace razonable usar Lambda en lugar de mantener una VM esperando eventos?

Una respuesta sólida debería relacionar ejecución corta, intermitencia, evento como disparador, ausencia de necesidad de control del sistema operativo y menor necesidad de administrar capacidad dedicada. “Porque es serverless” o “porque es más moderno” no explica la decisión.

Troubleshooting basado en evidencia

Síntoma	Qué revisar primero
Objeto existe pero Lambda no ejecuta	Región · trigger · prefix/suffix · permission de invocación · configuración S3
Lambda ejecuta pero no aparece log	Execution role · permisos de Logs · log group/stream · timestamp
Log aparece pero key no coincide	Objeto usado · evento · URL encoding del key
Aparecen dos invocaciones	Correlacionar evento; recordar semántica at-least-once antes de asumir fallo

> **Pregunta:** ¿Qué evidencia demuestra que el problema está en la integración y no en la existencia de la función?

7. Evidencias y criterios de aceptación

- Bucket privado + trigger S3 y función Lambda correctamente relacionados; el filtro se limita a incoming/*.json.
- Creación observable de incoming/order-001.json.
- Log de ejecución que contiene bucket, key y eventName correspondientes al mismo objeto.
- Evidencia de que el procesamiento no utiliza una VM dedicada ni modifica el baseline S01.
- Tabla comparativa EC2 vs Lambda completada con responsabilidades y trade-offs, no con respuestas absolutas.
- Cleanup validado.

Regla de evidencia: Una captura de que la función Lambda existe no constituye evidencia suficiente. El lab exige objeto → evento → ejecución → log.

8. Cost awareness y cleanup

El lab usa un bucket temporal, uno o pocos objetos pequeños, una función Lambda, pocas invocaciones y logs mínimos. No crea EC2, ALB, NAT Gateway, public IPv4, API Gateway, SQS, SNS ni Step Functions.

CONSERVAR COMO BASELINE

- dcl-dev-vpc · 10.20.0.0/16.
- dcl-dev-subnet-a y dcl-dev-subnet-b con sus AZ.
- Naming/tags persistentes del curso y evidencias del equipo.

ELIMINAR — cleanup obligatorio

21. Eliminar el trigger/event notification S3 asociado a Lambda.
22. Eliminar dcl-dev-order-event-s07.
23. Eliminar /aws/lambda/dcl-dev-order-event-s07.
24. Vaciar el bucket, eliminar objetos accidentales y eliminar el bucket.
25. Eliminar la inline policy del lab y después dcl-dev-role-lambda-s07.
26. Verificar que no quedaron recursos S07 ni modificaciones sobre S01.

Validación de cleanup: bucket, Lambda, trigger, log group y role S07 ya no existen; el baseline S01 permanece intacto.

9. Cierre técnico

- Serverless elimina la necesidad de administrar una VM dedicada para esta reacción; no elimina IAM, código, eventos, logs ni costo.
- Event-driven reduce el acoplamiento temporal entre productor y consumidor, pero introduce semántica de eventos y nuevas rutas de troubleshooting.
- Modernización es una decisión bajo restricciones; VM sigue siendo válida cuando las restricciones justifican mayor control o ejecución distinta.

Puente hacia S08: No escogemos VM, containers o serverless porque sean más modernos; los escogemos porque ciertas restricciones justifican sus responsabilidades y trade-offs. Ahora debemos defender la arquitectura completa bajo restricciones técnicas, operativas, de costo y negocio.

Fuentes oficiales

- AWS Lambda — Using an Amazon S3 trigger to invoke a Lambda function
- AWS Lambda — Process Amazon S3 event notifications with Lambda
- Amazon S3 — Event Notifications
- Amazon S3 — Object key name filtering
- AWS Lambda — Sending function logs to CloudWatch Logs